RAPPORT TECHNIQUE DE FIN DE PROJET (LIVRABLE E6)

Conception, Déploiement et Gouvernance du Data Warehouse VenteRapide

Auteur : Arailym PERNEBAY
Projet : Modern Data Stack & Enterprise Data Warehouse (VenteRapide)
Technologies : PostgreSQL 18, dbt Core 1.12, Python 3.11, SQL, Git
Compétences Validées : C16 (Modélisation DW), C17 (Historisation SCD), C18 (Gouvernance, Sécurité & Qualité)

1. Synthèse Exécutive & Présentation du Projet

1.1 Contexte et Objectifs Business

La société e-commerce **VenteRapide** a connu une croissance rapide, entraînant une fragmentation de ses données opérationnelles et des difficultés d'analyse décisionnelle. L'objectif principal de ce projet est de concevoir, d'automatiser et de sécuriser un **Data Warehouse d'entreprise** centralisé.

Le Data Warehouse répond à trois enjeux majeurs :

1. **Performance Analytique (C16) :** Structure schématique en étoile (Star Schema) selon la méthodologie de Ralph Kimball pour accélérer les requêtes BI.
2. **Fidélité Historique (C17) :** Prise en charge des évolutions temporelles des attributs clients (*Slowly Changing Dimensions* Type 2) pour préserver la vérité exacte au moment de chaque commande (*Point-in-Time Join*).
3. **Gouvernance et Sécurité (C18) :** Cloisonnement strict des accès via Role-Based Access Control (RBAC), conformité RGPD (Registre Article 30 et anonymisation) et observabilité de la qualité des données via assertions dbt.

1.2 Stack Technique et Architecture Global

L'architecture adopte un modèle **ELT moderne** (*Extract, Load, Transform*) où les données brutes sont chargées dans PostgreSQL avant d'être transformées par dbt.

```
None
[Sources / Seeds] ──> [Couche Staging (stg_*)] ──> [Snapshots (scd_*)] ──> [Couche Marts (dim_*, fact_*)]

|

└──> BI / Reporting (Read-Only)
```

2. Modélisation Dimensionnelle & Architecture Data (Compétence C16)

2.1 Structuration de la Modélisation en Étoile

Conformément aux normes Kimball, la modélisation sépare strictement les faits mesurables et les dimensions contextuelles.

- **Table de Faits Centrales (`fact_commandes`, `fact_retours`) :** Enregistre les métriques transactionnelles (`montant_total`, `frais_livraison`, `delai_expedition_jours`, `nb_articles`). Chaque ligne représente un événement métier granulaire relié aux dimensions par des clés substitutives (*surrogate keys*).
- **Dimensions Structurantes :**
  - `dim_produit` : Catalogue produit avec hiérarchie des catégories (Modélisation SCD Type 1 pour écrasement des mises à jour catalogue).
  - `dim_date` : Dimension calendrier complète permettant des agrégations temporelles fines (jour, mois, trimestre, indicateur week-end/jour férié).
  - `scd_client` : Dimension client historisée en SCD Type 2.

2.2 Normalisation dans la Couche Staging

Chaque vue de staging (`models/staging/stg_*.sql`) effectue le nettoyage initial :

- Typage strict des données (moulage des dates, entiers и décimaux).
- Standardisation du nommage des colonnes en snake_case.
- Filtrage des enregistrements invalides ou corrompus.

3. Gestion de l'Historique & SCD Type 2 (Compétence C17)

3.1 Implémentation du Snapshotted Client (`snapshots/scd_client.sql`)

Pour suivre les déménagements et changements de segment des clients sans perdre l'état historique, dbt snapshot génère automatiquement les fenêtres de validité temporelle (`dbt_valid_from` et `dbt_valid_to`).

```sql
{% snapshot scd_client %}
{{
    config(
      target_schema='public',
      unique_key='client_id',
      strategy='check',
      check_cols=['ville', 'segment', 'code_postal']
    )
}}
select * from {{ ref('stg_clients') }}
{% endsnapshot %}
```

3.2 Requête d'Association Temporelle (Point-in-Time Join)

La jointure temporelle permet d'associer une commande passée à une date T avec l'adresse du client exacte à la date T (et non sa ville actuelle).

```sql
SELECT
    fc.commande_id,
    fc.client_id,
    dd.date_jour AS date_commande,
    sc.ville AS ville_au_moment_de_la_commande,
    sc.segment
FROM public.fact_commandes fc
JOIN public.dim_date dd ON fc.date_commande_id = dd.date_id
LEFT JOIN public.scd_client sc ON fc.client_id = sc.client_id
   AND ((dd.date_jour >= sc.dbt_valid_from::date
         AND (dd.date_jour < sc.dbt_valid_to::date OR sc.dbt_valid_to IS NULL))
        OR sc.dbt_valid_to IS NULL);
```

4. Gouvernance, Sécurité, RGPD & Qualité (Compétence C18)

4.1 Politique de Sécurité RBAC (*Principle of Least Privilege*)

Trois rôles étanches sont définis sur la base de données PostgreSQL :

Rôle	**Portée & Usage**	**Privilèges Attribués**
`postgres` / `admin`	Administration système & DDL	`SUPERUSER` / Droits complets
`dbt_user`	Agent d'exécution des pipelines ELT	`CREATE`, `SELECT`, `INSERT`, `UPDATE` sur le schéma `public`
`reporting`	Outils de Business Intelligence / Analysts	`USAGE` sur schéma, `SELECT` uniquement sur les tables/vues finalisées

4.2 Conformité RGPD & Protection des Données Personnelles

En application de la réglementation européenne, la gouvernance inclut :

1. **Registre des Traitements (Article 30) :** Inventaire consigné dans `docs/GESTION_ACCES_ET_RGPD.md` détaillant la finalité, la durée de conservation et la minimisation des données.
2. **Procédure d'Anonymisation :** Scripts d'anonymisation des données clients inactifs (hachage des e-mails et suppression des identifiants directs au-delà de la durée légale de rétention).

4.3 Cadre de Test et Observabilité

Plus de 70 assertions automatiques dbt contrôlent l'intégrité de la chaîne :

- Tests d'unicité et de non-nullité sur l'ensemble des clés primaires.
- Validation des relations d'intégrité référentielle entre faits et dimensions.
- Génération du graphe d'orienté acyclique (DAG de lignage) assurant la traçabilité complète des données.

5. Organisation du Projet & Rapport d'Audit

5.1 Matrice d'Organisation (WBS & Planification)

Le projet a été exécuté en 4 phases techniques structurées :

- **Phase 1 :** Infrastructure, Scripts de Backup (`pg_dump`) et Logs (`postgresql.conf`).
- **Phase 2 :** Ingestion Staging, Modélisation Marts en Étoile et Tests Qualité dbt.
- **Phase 3 :** Configuration Snapshots SCD2, Jointure Temporal Point-in-Time, Modèle SCD1.
- **Phase 4 :** Sécurisation RBAC, Déploiement RGPD, Documentation Lineage DAG et Audit.

5.2 Index des Preuves d'Audit (Screenshots Report)

Tâche / Compétence	**Description de la Preuve**	**Fichier Screenshot (`screenshots_report/`)**
Task 3.1 (C16)	Connexion et validation de l'environnement dbt	`01_dbt_connection_debug.png`
Task 3.2 (C16)	Ingestion des seeds et exécution des vues de staging	`02_run_stg_retours_sql_view_model&dbt_seed.png`
Task 3.3 (C16)	Compilation et exécution du modèle de faits retours	`03_run_output_sql_view_model_fact_retours.png`
Task 3.4 (C18)	Rapport d'exécution des tests dbt sur les faits	`04_test_output_fact_retours.png`
Task 3.4 (C16)	Consultation SQL des enregistrements du mart `fact_retours`	`05_local_dbt_fact_retours.png`
Task 3.4 (C16)	Consultation SQL des données nettoyées `stg_retours`	`06_local_dbt_stg_retours.png`
Task 3.5 (C18)	Test des droits restreints de l'utilisateur `reporting` (Partie 1)	`07_b_task_3_5_readonly_test.png`
Task 3.5 (C18)	Refus d'écriture/modification pour le rôle read-only (Partie 2)	`08_task_3_5_readonly_test.png`

Tâche / Compétence	Description de la Preuve	Fichier Screenshot (`screenshots_report/`)
Task 3.6 (C17)	Résultat de la requête de jointure temporelle Point-in-Time	`09_task_3_6_c17_point_in_time_fact_scd_join.png`
Task 3.6 (C17)	Vérification de l'historisation SCD2 (lignes actives vs expirées)	`10_task_3_6_c17_scd2_client_history_tracking.png`
Task 3.7 (C18)	Graphe de lignage interactif dbt (Lineage DAG)	`11_task_3_7_c18_dbt_lineage_dag_graph.png`

---

6. Conclusion & Perspective de Scalabilité

Le Data Warehouse **VenteRapide** est désormais fully-functional, sécurisé et prêt pour la production. L'architecture dbt permet une scalabilité horizontale aisée : l'ajout de nouveaux Data Marts (ex: Marketing ou Logistique) s'effectue par simple création de modèles SQL dbt sans impacter la structure existante.